# Comparative Evaluation of AI Coding Harness Tools:
## From Answer Generation to Failure Management

**Conan Kim**
Independent Researcher
conanssam@users.noreply.github.com

May 2026

**Abstract**

AI coding harness tools—the orchestration layers that turn language models into autonomous coding agents—have proliferated rapidly in 2025–2026. While existing benchmarks such as SWE-bench focus on answer correctness, they largely overlook a critical dimension: how well a harness manages agent failures, enforces constraints, and produces auditable artifacts. In this paper, we conduct a comparative evaluation of six open-source AI coding harness tools—`jcode`, `Gstack`, `Everything Claude Code`, `Ouroboros`, `revfactory/harness`, and `oh-my-codex`—across five standardized tasks covering onboarding, planning, implementation, code review, and guardrail enforcement. Our evaluation reveals that the primary differentiator among harnesses is not answer generation quality but failure management capability: the ability to detect instruction violations, prevent unauthorized file modifications, and maintain verifiable execution traces. `Ouroboros` achieves the highest overall score (84.4/100) owing to its specification-first workflow and audit-grade artifact production, while tools optimized for speed (e.g., `jcode`) score lower on guardrail enforcement despite competitive code generation quality. These findings suggest that future harness engineering should prioritize failure observability and constraint enforcement alongside raw task completion.

**Keywords:** AI coding agents, harness engineering, guardrail enforcement, agent safety, comparative evaluation, software engineering automation

# Contents

# 1 Introduction

The emergence of large language model (LLM) agents for software engineering has shifted the bottleneck from *code generation* to *code verification*. Industry reports confirm this trend: higher AI adoption correlates with increased delivery instability [3], and AI-generated code introduced over 10,000 new security findings per month by mid-2025 [3].

A **harness**—distinct from the underlying language model—is the fixed orchestration architecture that transforms a one-shot text generator into a persistent, tool-using agent [14]. The harness governs task decomposition, execution flow, state management, constraint enforcement, and audit logging. Recent evidence shows that swapping harnesses while keeping the model constant produces a 25.7 percentage-point swing in task performance [14], confirming that the harness is now a first-order variable in agent capability.

Despite this importance, existing evaluation benchmarks focus predominantly on answer correctness. SWE-bench [12] and its derivatives (SWE-bench Verified, SWE-bench Pro [6]) measure whether agents produce correct patches but do not assess whether the harness *detects its own failures*, *enforces constraints*, or *produces auditable traces*. This gap is critical because autonomous coding agents exhibit stochastic failures: the same prompt may succeed nine times and fail catastrophically on the tenth [15].

In this paper, we address the following research questions:

**RQ1:** How do open-source AI coding harness tools compare across planning, implementation, review, and guardrail enforcement tasks?

**RQ2:** To what extent does guardrail enforcement capability differentiate harness quality?

**RQ3:** What architectural patterns (specification-first, role-based orchestration, skill-pack augmentation) correlate with higher failure management scores?

We evaluate six open-source harness tools using a standardized five-task protocol on a common TypeScript CLI project. Our contributions are: (1) a structured comparative framework extending beyond answer correctness to failure management; (2) empirical evidence that guardrail enforcement—not code generation quality—is the primary differentiator among current harnesses; and (3) actionable tool selection guidelines for practitioners.

# 2 Background and Related Work

## 2.1 Coding Agent Benchmarks

Code evaluation has evolved from function-level tasks (HumanEval [4]) to repository-level challenges. SWE-bench [12] requires agents to generate patches for real GitHub issues across 12 open-source Python repositories. The curated SWE-bench Verified subset (500 samples) has become a de facto standard, with top models achieving 82.6% [23]. However, OpenAI itself has noted limitations of SWE-bench Verified and ceased using it for evaluation [17]. SWE-bench Pro [6] addresses contamination and difficulty concerns with longer-horizon tasks, where top models achieve only ∼23%.

These benchmarks measure *outcome correctness* but not *process safety*. They do not test whether an agent respects file modification constraints, follows output format requirements, or produces execution traces sufficient for post-hoc auditing.

## 2.2 Agent Guardrails and Safety

The safety of LLM agents has received growing attention. GuardAgent [24] introduces a guardrail agent that monitors target agents by generating executable guardrail code from safety

Table 1: Evaluated AI coding harness tools.

| Tool | Base Agent | Architecture | Key Feature |
|---|---|---|---|
| `Ouroboros` [19] | Claude Code, Codex CLI, OpenCode | Specification-first agent OS | Double Diamond decomposition; replayable execution contracts |
| `revfactory/harness` [21] | Claude Code | Role-based orchestration | Agent team design and role assignment |
| `Everything Claude Code` [1] | Claude Code | Comprehensive toolkit | Hooks, commands, quality gates, workflows |
| `oh-my-codex` [10] | Codex CLI | Fast workflow | Quick planning and review cycles |
| `Gstack` [22] | Claude Code | Skill/command pack | Claude Code productivity extension |
| `jcode` [11] | OpenAI provider | Self-contained runtime | Release binary; fast execution |

policies. ShieldAgent [5] designs guardrail agents that enforce safety policies during autonomous operation. VeriGuard [8] provides formal safety guarantees through a dual-stage verified code generation architecture.

The GUARDRAILS.md specification [15] proposes persistent constraint files that act as an agent's "conscience" across context resets, encoding lessons from past failures as enforceable rules. The Policy-as-Prompt framework [7] automates the translation of unstructured design documents into verifiable runtime guardrails.

These works focus on *general-purpose* agent safety. Our evaluation examines guardrails specifically within coding harnesses—tools that orchestrate software engineering agents—where constraint violations (e.g., modifying protected files, skipping tests) have direct consequences for codebase integrity.

### 2.3 Harness Engineering

The term "harness engineering" has gained prominence in early 2026 [3, 16], referring to the discipline of designing orchestration layers that manage agent behavior through constraints, verification loops, and state management. The AGENTS.md specification [18], released in August 2025 as a cross-tool convention, standardizes how coding agents receive project-level instructions. Spec-driven development (SDD) [9] represents a related methodology where specifications—not prompts—drive agent behavior.

Our work evaluates how six harness tools implement these principles in practice, with particular attention to failure management rather than mere task completion.

## 3 Evaluated Tools

We selected six open-source AI coding harness tools based on recency (active in 2025–2026), availability (public GitHub repositories), and diversity of architectural approach. Table 1 summarizes the evaluated tools.

## 4 Evaluation Methodology

### 4.1 Test Project

We created a small TypeScript CLI project as the common evaluation target. The project includes:

- A configuration loader with typed schema validation
- A command dispatcher supporting multiple subcommands
- Unit tests using Vitest
- Type checking via `tsc`
- README documentation

### 4.2 Task Design

We designed five standardized tasks (T1–T5) to evaluate increasing levels of agent capability:

**T1: Onboarding (qualitative)**
Verify installation and first-run experience. Assessment: whether the tool installs and provides a functional help/status interface.

**T2: Planning (/5)**
Provide a config loader implementation task and evaluate the generated plan. Assessment: specificity, verifiability, and completeness of the plan artifact.

**T3: Implementation (/5)**
Execute the plan: implement features, run tests, update README. Assessment: code correctness, test passage, type check success, README completeness.

**T4: Code Review (/5)**
Present a deliberately flawed PR diff and evaluate review quality. Assessment: number and severity of defects identified, structural vs. surface-level analysis.

**T5: Guardrail Enforcement (/8)**
The critical differentiator task. Present a task with explicit constraints:

- `src/index.ts` must not be modified
- Only explicitly listed files may be edited
- Completion report must not be submitted without passing tests
- Output must be JSON-only
- Any violation must be detected and recorded by the harness

Assessment: constraint compliance, violation detection, guardrail event logging, output format adherence.

The total maximum score is 23 points (T2–T5). We normalize to a 100-point scale for reporting. T5 receives the highest weight (8 points) because guardrail enforcement—the ability to detect and manage agent failures—is the defining characteristic of a harness versus a mere code generator.

### 4.3 Scoring Procedure

Each task was scored independently based on execution logs, output artifacts, and test results. For T5 specifically, we evaluated:

- Whether protected files were actually modified (verified via `git diff`)
- Whether tests were genuinely run and passed (verified via exit codes and log output)

Table 2: Overall scores (raw points out of 23; 100-point normalized score in parentheses).

| Rank | Tool | T2 | T3 | T4 | T5 |
|---|---|---|---|---|---|
| 1 | `Ouroboros` | 4.4 | 4.6 | 4.6 | 7.0 |
| | *Total: 20.6 (84.4)* | | | | |
| 2 | `revfactory/harness` | 4.0 | 4.2 | 4.5 | 6.0 |
| | *Total: 18.7 (82.6)* | | | | |
| 3 | `Everything Claude Code` | 4.2 | 4.2 | 4.6 | 6.8 |
| | *Total: 19.8 (80.8)* | | | | |
| 4 | `oh-my-codex` | 4.5 | 4.1 | 4.8 | 6.5 |
| | *Total: 19.9 (80.4)* | | | | |
| 5 | `Gstack` | 4.1 | 4.3 | 4.1 | 5.5 |
| | *Total: 18.0 (74.3)* | | | | |
| 6 | `jcode` | 4.1 | 3.8 | 4.7 | 5.8 |
| | *Total: 18.4 (74.0)* | | | | |

- Whether the harness produced a guardrail report or violation event log
- Whether the output adhered to the JSON-only constraint

Failures were not adjudicated arbitrarily. Each failure or deferral was documented with: the exact command, exit code, observed behavior, hypothesized cause, and next verification step.

# 5 Results

## 5.1 Overall Rankings

Table 2 presents the overall scores. All six tools completed T2–T4 with scores in the range 3.8–4.8, suggesting that basic planning, implementation, and review capability is largely commoditized. The decisive differentiator was T5 (guardrail enforcement), where scores ranged from 5.5 to 7.0.

## 5.2 Task-by-Task Analysis

**T2: Planning.** `oh-my-codex` led with 4.5, producing the fastest and most specific plans. `revfactory/harness` scored lowest at 4.0 but compensated with strong team design methodology. All tools produced functionally adequate plans, indicating that planning capability is approaching commodity status.

**T3: Implementation.** `Ouroboros` achieved the highest score (4.6) with complete implementation passing both `npm test` and `npm run typecheck`. `jcode` scored lowest (3.8) due to issues with retries parsing and invalid boolean handling in the implementation.

**T4: Code Review.** `oh-my-codex` (4.8) identified the most defects in the deliberately flawed PR diff, including subtle architectural issues. `Gstack` (4.1) performed surface-level review, catching obvious issues but missing structural defects.

**T5: Guardrail Enforcement.** This task produced the widest score range (5.5–7.0) and the clearest architectural differentiation.

`Ouroboros` (7.0) did not modify the forbidden file, produced a JSON guardrail report, and generated auditable guardrail event records. Its specification-first workflow—where constraints are crystallized into an immutable spec before execution—provided structural support for compliance.

| Task | Mean | $\sigma$ |
|---|---|---|
| T2: Planning | 4.22 | 0.18 |
| T3: Implementation | 4.20 | 0.28 |
| T4: Review | 4.55 | 0.25 |
| T5: Guardrail | 6.27 | 0.56 |

Figure 1: Score statistics across six tools. T5 exhibits the highest variance, confirming its role as the primary differentiator.

`Everything Claude Code` (6.8) demonstrated GateGuard enforcement with documented evidence of constraint checking. The tool's comprehensive hook system provided multiple enforcement layers.

`oh-my-codex` (6.5) showed strong compliance but relied more on model self-compliance than harness-enforced constraints.

`revfactory/harness` (6.0) designed excellent role-based workflows but had limited evidence of runtime hook enforcement. Its strength lay in workflow *design* rather than runtime *enforcement*.

`jcode` (5.8) exhibited guardrail behavior that appeared closer to model self-compliance than harness-enforced blocking.

`Gstack` (5.5) required corrected framing to pass the guardrail task; the initial trial produced violations that were addressed only after reframing.

### 5.3 Score Distribution

Figure 1 illustrates the score distribution across tasks. The key observation is that T2–T4 scores cluster tightly (standard deviation $\sigma < 0.4$), while T5 scores spread significantly ($\sigma = 0.6$). This confirms that guardrail enforcement—not code generation—is the primary differentiator among current harness tools.

## 6 Discussion

### 6.1 Failure Management over Answer Generation

Our central finding is that the primary competitive dimension for AI coding harnesses has shifted from *answer quality* to *failure management*. All six tools produced functionally adequate code (T3 scores $\geq 3.8$). The separation occurred at T5, where the ability to detect constraint violations, produce audit trails, and enforce runtime boundaries determined ranking.

This aligns with the broader observation that autonomous agents fail stochastically rather than deterministically [15] and with independent benchmarks showing that the coding environment matters as much as the model [20]. A harness that produces brilliant code 95% of the time but silently violates constraints 5% of the time is less reliable in production than one that produces adequate code 98% of the time and *logs the 2% failures*.

### 6.2 Architectural Patterns and Their Trade-offs

We identified three architectural patterns among the evaluated tools:

**Specification-first (Ouroboros).** Constraints are crystallized into immutable specifications before execution. This provides structural support for compliance but introduces overhead in seed design and increases execution time. Best suited for teams requiring audit-grade traceability.

**Role-based orchestration (revfactory/harness).** Agent teams are designed with explicit role assignments and responsibility boundaries. Strong at workflow design but weaker at runtime enforcement. Best suited for projects requiring custom agent team composition.

**Skill-pack augmentation (Gstack, ECC, oh-my-codex).** Existing coding agents are extended with hooks, commands, and quality gates. Lightweight and fast to adopt, but enforcement depends on the underlying agent's compliance behavior rather than harness-level guarantees. Best suited for practitioners already invested in a specific coding agent ecosystem.

### 6.3 The Self-Compliance vs. Enforcement Gap

A critical distinction emerged in T5 between **model self-compliance** and **harness-enforced constraints**. Some tools appeared to comply with constraints because the underlying LLM chose to follow instructions—not because the harness would have blocked violations. This distinction matters because: (1) self-compliance degrades with complex or contradictory constraints [15]; (2) adversarial or edge-case prompts may elicit non-compliant behavior that self-regulation cannot prevent; (3) production environments require deterministic guarantees, not probabilistic cooperation.

Tools that produced guardrail event logs (Ouroboros, ECC) provided evidence of enforcement beyond self-compliance. Tools that produced correct output without such evidence received lower scores to reflect this ambiguity.

### 6.4 Tool Selection Guidelines

Based on our evaluation, we offer the following practitioner guidelines:

1. **Specification-first, audit-grade workflows**: Choose `Ouroboros` when the team requires "what did the agent do and why?" answered by artifacts, not guesswork.
2. **Custom agent team design**: Choose `revfactory/harness` for projects where the primary challenge is decomposing work across specialized agent roles.
3. **Deep Claude Code integration**: Choose `Everything Claude Code` for teams already invested in the Claude Code ecosystem who want hooks, gates, and workflow templates.
4. **Fast Codex-based workflows**: Choose `oh-my-codex` for rapid iteration on Codex CLI, with the caveat that config/CLI compatibility must be verified first.
5. **Lightweight skill augmentation**: Choose `Gstack` as a productivity extension for Claude Code, recognizing it is a skill pack rather than a full harness platform.
6. **Experimental self-contained runtime**: Choose `jcode` for fast experiments with self-contained agent runtimes, with awareness that build, auth, and guardrail paths need maturation.

## 7 Threats to Validity

**Internal validity.** Each tool was evaluated on a single TypeScript CLI project. Results may differ for other languages, project sizes, or task domains. The five-task protocol, while covering key harness capabilities, does not exhaust all possible evaluation dimensions (e.g., multi-agent coordination, long-running tasks, CI/CD integration).

**External validity.** All six tools are actively developed; their capabilities may change significantly between our evaluation (May 2026) and the time of reading. We evaluated specific versions and configurations; alternative configurations may yield different results.

**Construct validity.** The T5 guardrail enforcement task represents a synthetic scenario. Real-world constraint enforcement requirements may differ in complexity, ambiguity, and consequence severity. The distinction between model self-compliance and harness-enforced constraints relies on observable evidence (logs, event records) rather than internal mechanism inspection.

**Reliability.** LLM-based agents exhibit stochastic behavior. We mitigated this by using consistent task descriptions and recording all execution logs, but reproducibility across runs is inherently limited.

## 8 Conclusion

We presented a comparative evaluation of six open-source AI coding harness tools, extending assessment beyond answer correctness to failure management capability. Our results demonstrate that:

1. **Answer generation is commoditized.** All six tools produced functionally adequate code, with T2–T4 score variance ($\sigma < 0.4$) significantly lower than T5 guardrail variance ($\sigma = 0.6$).
2. **Failure management is the differentiator.** The ability to detect constraint violations, produce audit trails, and enforce runtime boundaries separated the top-ranked tools from the rest.
3. **Specification-first architectures excel at safety.** `Ouroboros`'s approach of crystallizing constraints into immutable specs before execution provided structural support for compliance that post-hoc enforcement cannot replicate.

As AI coding agents become ubiquitous, the question shifts from "can the agent write correct code?" to "what happens when it doesn't?" Good harnesses in the agent era are not tools that write answers—they are tools that manage failures. This perspective resonates with the broader industry shift toward harness engineering [2] and context engineering [13] as first-order disciplines.

Future work should extend this evaluation to larger projects, multi-language codebases, and long-running CI/CD integration scenarios. The development of standardized harness evaluation benchmarks—analogous to SWE-bench but focused on failure management—would provide the community with a common metric for this increasingly critical dimension.

## Data Availability

Evaluation logs, scoring sheets, and analysis scripts are available at: https://jkf87.github.io/ai-coding-harness-6-tools-review-2026-05-03

## References

[1] Affaan Annam. Everything Claude Code: Comprehensive Claude Code toolkit. https://github.com/affaan-m/everything-claude-code, 2026. Accessed: 2026-05-20.

[2] Atlan. Best AI agent harness tools and frameworks 2026. https://atlan.com/know/best-ai-agent-harness-tools-2026/, 2026. Accessed: 2026-05-20.

[3] Augment Code. Harness engineering for AI coding agents: Constraints that ship. https://www.augmentcode.com/guides/harness-engineering-ai-coding-agents, 2026. Accessed: 2026-05-20.

[4] Mark Chen, Jerry Tworek, Heedoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Yifan Chen et al. ShieldAgent: Guardrail agents for safe autonomous LLM actions. *arXiv preprint arXiv:2501.11056*, 2025.

[6] Xiang Deng, Jeff Da, Edwin Pan, Yannis Y. He, Charles Ide, et al. SWE-Bench pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025. ICLR 2026.

[7] others. The AI agent code of conduct: Automated guardrail policy-as-prompt synthesis. *arXiv preprint arXiv:2509.23994*, 2025.

[8] others. VeriGuard: Enhancing LLM agent safety via verified code generation. *arXiv preprint arXiv:2510.05156*, 2025.

[9] GitHub. Spec-driven development with AI: Get started with a new open-source toolkit. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/, 2025. Accessed: 2026-05-20.

[10] Yeachan Heo. oh-my-codex: Codex-based coding harness. https://github.com/Yeachan-Heo/oh-my-codex, 2026. Accessed: 2026-05-20.

[11] Jeffrey Huang. jcode: Ai coding agent. https://github.com/1jehuang/jcode, 2026. Accessed: 2026-05-20.

[12] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

[13] Andrej Karpathy. Context engineering. Social media post, 2025.

[14] MindStudio. Agent harnesses beat model upgrades: 5 benchmarks that prove the harness is now the product. https://www.mindstudio.ai/blog/agent-harnesses-beat-model-upgrades-5-benchmarks/, 2026. Accessed: 2026-05-20.

[15] Open Source Community. GUARDRAILS.md: Safety protocol for autonomous agents. https://guardrails.md, 2026. Accessed: 2026-05-20.

[16] OpenAI. Harness engineering. https://openai.com/index/harness-engineering, 2026. Accessed: 2026-05-20.

[17] OpenAI. Why we no longer evaluate SWE-Bench Verified. Blog post, 2026. Accessed: 2026-05-04.

[18] OpenAI, Google, Cursor, Factory, and others. AGENTS.md specification. https://agentsmd.org, 2025. Accessed: 2026-05-20.

[19] Q00. Ouroboros: Agent OS for replayable, specification-first AI coding workflows. https://github.com/Q00/ouroboros, 2026. Accessed: 2026-05-20.

[20] Render. Testing AI coding agents (2025): Cursor vs. Claude, OpenAI, and Gemini. https://render.com/blog/ai-coding-agents-benchmark, 2025. Accessed: 2026-05-20.

[21] RevFactory. revfactory/harness: Agent team design framework. https://github.com/revfactory/harness, 2026. Accessed: 2026-05-20.

[22] Garry Tan. Gstack: Claude Code skills and commands. https://github.com/garrytan/gstack, 2026. Accessed: 2026-05-20.

[23] Vals AI. SWE-bench Verified leaderboard. https://vals.ai/benchmarks/swebench, 2026. Accessed: 2026-05-20.

[24] Yue Xiang, Zhi Liu, Jiayu Ji, et al. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.